

LiveMatch – Panel Wyników Piłkarskich na Żywo

Dokumentacja końcowa projektu

Jakub Urbański

ZTI – Informatyka Stosowana
Rok akademicki 2025/2026

Repozytorium projektu

<https://github.com/ur8an5ky/LiveMatch>

Spis treści

1	Wprowadzenie	4
1.1	Cel projektu	4
1.2	Problematyka domeny	4
1.3	Zakres dokumentu	4
2	Analiza wymagań	5
2.1	Wymagania funkcjonalne	5
2.2	Wymagania niefunkcjonalne	5
2.3	Diagram przypadków użycia	6
3	Stos technologiczny	7
3.1	Backend	7
3.2	Frontend	7
3.3	Infrastruktura	8
4	Architektura systemu	9
4.1	Architektura wdrożeniowa	9
4.2	Architektura logiczna	9
4.3	Przepływ danych	10
5	Model danych	11
5.1	Diagram klas modelu domeny	11
5.2	Diagram ERD	12
5.3	Inicjalizacja danych	13
6	Warstwa serwerowa	14
6.1	Kontrolery REST	14
6.2	Warstwa serwisów	14
6.3	Repozytoria Spring Data JPA	14
6.4	Walidacja i obsługa wyjątków	15
7	Bezpieczeństwo	16
7.1	Architektura JWT	16
7.2	Konfiguracja Spring Security	17
7.3	Hashowanie haseł	17
8	Programowanie aspektowe	18
8.1	Idea	18
8.2	Adnotacja @Auditable	18
8.3	Aspekt AuditAspect	18
8.4	Encja AuditLog	18
8.5	Schemat działania	18
9	Komunikacja w czasie rzeczywistym	19
9.1	WebSocket i STOMP	19
9.2	Konfiguracja serwera	19
9.3	Kanały tematyczne	19
9.4	Publikacja zdarzeń	19
9.5	Subskrypcja po stronie klienta	20

9.6	Diagram sekwencji – dodanie zdarzenia	21
10	Frontend	22
10.1	Routing i kontrola dostępu	22
10.2	Zarządzanie stanem	22
10.3	Komunikacja z API	22
10.4	Internacjonalizacja	22
10.5	Komponenty UI	22
11	Konteneryzacja i wdrożenie	23
11.1	Multi-stage build backendu	23
11.2	Multi-stage build frontendu	23
11.3	Konfiguracja nginx	23
11.4	Kompozycja Docker Compose	23
11.5	Diagram wdrożeniowy	24
12	Testy	25
12.1	Wybrane testy jednostkowe	25
12.2	Generowanie raportu	25
13	Podręcznik użytkownika	26
13.1	Logowanie	26
13.2	Widok kibica – lista meczów	27
13.3	Widok kibica – szczegóły meczu	28
13.4	Panel administratora	29
13.5	Sterowanie meczem na żywo	30
13.6	Dodawanie zdarzenia	31
13.7	Zmiana języka i motywu	32
14	Informacja wdrożeniowa	33
14.1	Wymagania środowiska	33
14.2	Procedura wdrożenia	33
14.3	Zatrzymanie i sprzątanie	33
14.4	Domyślne dane logowania	33
15	Wnioski	34
15.1	Realizacja wymagań projektowych	34
15.2	Wyzwania i decyzje projektowe	34
15.3	Kierunki rozwoju	34

1 Wprowadzenie

1.1 Cel projektu

Głównym celem projektu *LiveMatch* jest stworzenie interaktywnej platformy webowej umożliwiającej śledzenie wyników i zdarzeń z meczów piłkarskich w czasie rzeczywistym. Aplikacja adresowana jest do dwóch grup użytkowników: administratorów (sprawozdawców sportowych), odpowiedzialnych za wprowadzanie zdarzeń boiskowych, oraz kibiców, obserwujących przebieg meczu z poziomu przeglądarki. System rozwiązuje problem opóźnień w dostarczaniu informacji sportowych, eliminując konieczność ręcznego odświeżania strony w celu poznania aktualnego stanu rozgrywki.

1.2 Problematyka domeny

Tradycyjne portale sportowe wykorzystują często mechanizm *long polling* lub okresowych odświeżeń, co generuje niepotrzebny narzut komunikacyjny i wprowadza opóźnienia odczuwalne przez użytkownika. Wybrana w projekcie technologia WebSocket – w połączeniu z protokołem warstwy aplikacji STOMP – pozwala utrzymać trwałe, dwukierunkowe połączenie między serwerem a klientem. Gdy administrator wprowadzi nowe zdarzenie, jest ono natychmiastowo rozsyłane do wszystkich subskrybentów odpowiedniego kanału (mecz), bez konieczności inicjowania nowego żądania HTTP przez klienta.

1.3 Zakres dokumentu

Dokumentacja obejmuje następujące zagadnienia: analizę wymagań funkcjonalnych i нефункциональных, uzasadnienie wyboru wykorzystanych technologii, architekturę systemu i model danych, opis kluczowych aspektów implementacyjnych (bezpieczeństwo, komunikacja asynchroniczna, programowanie aspektowe), proces konteneryzacji i wdrożenia, podsumowanie wykonanych testów, podręcznik użytkownika końcowego oraz instrukcję uruchomieniową.

2 Analiza wymagań

2.1 Wymagania funkcjonalne

Aplikacja realizuje funkcjonalności podzielone według ról użytkowników:

Funkcje dostępne dla wszystkich (kibic):

- Przeglądanie listy meczów z filtracją po statusie (zaplanowane, trwające, zakończone).
- Wyświetlanie szczegółów wybranego meczu wraz z osią czasu zdarzeń.
- Wyświetlanie powiadomień o zdarzeniach trwającego meczu w czasie rzeczywistym.
- Przełączanie języka interfejsu (polski / angielski).
- Przełączanie motywu kolorystycznego (jasny / ciemny).

Funkcje dostępne wyłącznie dla administratora:

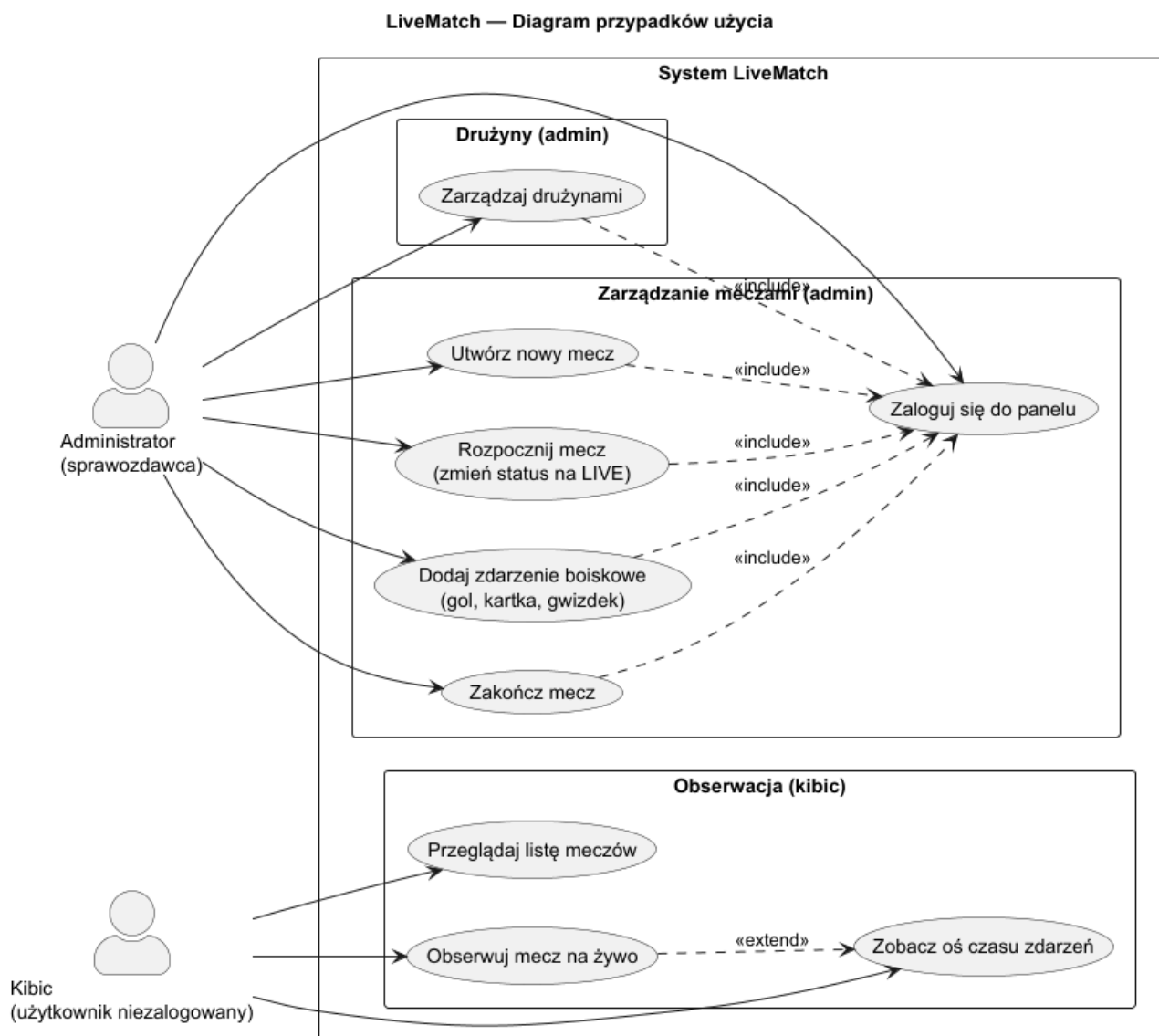
- Logowanie i wylogowywanie z systemu (autentykacja JWT).
- Zarządzanie drużynami: tworzenie, edycja, usuwanie – z obsługą zarówno klubów (z nazwą), jak i reprezentacji narodowych (identyfikowanych kodem ISO państwa).
- Zarządzanie meczami: planowanie, edycja, usuwanie.
- Sterowanie statusem meczu (zaplanowany → trwający → zakończony | odwołany) z walidacją dozwolonych przejść.
- Rejestrowanie zdarzeń boiskowych podczas trwającego meczu (gol, kartka, zmiana, koniec połowy, gwizdek końcowy).

2.2 Wymagania niefunkcjonalne

- **Natychmiastowość:** zdarzenia muszą docierać do klientów w czasie krótszym niż 500 ms od ich wprowadzenia przez administratora.
- **Skalowalność:** architektura musi pozwalać na obsługę dziesiątek równoległych obserwacji jednego meczu.
- **Bezpieczeństwo:** operacje modyfikujące dane muszą być dostępne wyłącznie dla uwierzytelnionych administratorów. Hasła przechowywane w formie zhashowanej.
- **Portabilność:** aplikacja musi być w pełni skonteneryzowana i uruchamialna na dowolnej maszynie z Dockerem, bez ręcznej konfiguracji środowiska.
- **Internacjonalizacja:** wszystkie elementy interfejsu (etykiety, komunikaty, daty) muszą być przygotowane do tłumaczenia.

2.3 Diagram przypadków użycia

Funkcjonalności systemu z perspektywy obu aktorów przedstawia diagram przypadków użycia (rys. 1). Wszystkie akcje administracyjne są poprzedzone relacją «include» do logowania.



Rysunek 1: Diagram przypadków użycia systemu LiveMatch

3 Stos technologiczny

3.1 Backend

Java 21 i Spring Boot 3.5.14. Wybór platformy Spring Boot wynika z bogatego ekosystemu modułów (Web, Security, Data, WebSocket, AOP) oraz konwencji *convention over configuration*, znacznie skracającej czas implementacji typowych funkcjonalności. Java 21 zapewnia dostęp do nowoczesnych konstrukcji języka – *records*, *pattern matching*, *sealed classes* – które wykorzystano w warstwie DTO.

Spring Data JPA + Hibernate. Warstwa dostępu do danych zrealizowana w oparciu o odwzorowanie obiektowo-relacyjne. Repozytoria są deklarowane jako interfejsy rozszerzające `JpaRepository`, dzięki czemu większość zapytań CRUD nie wymaga ręcznej implementacji. W konfiguracji ustawiono `spring.jpa.hibernate.ddl-auto=update`, co pozwala Hibernate’owi automatycznie aktualizować schemat bazy na podstawie deklaracji encji.

Spring Security + JWT 0.12.6. Bezpieczeństwo aplikacji oparto na tokenach JWT (JSON Web Tokens) w trybie *stateless* – serwer nie przechowuje sesji użytkownika, każde żądanie jest autoryzowane na podstawie nagłówka `Authorization: Bearer <token>`. Hasła przechowywane są w bazie w postaci zhashowanej algorytmem BCrypt.

Spring WebSocket + STOMP. Komunikacja w czasie rzeczywistym wykorzystuje protokół STOMP (Simple Text Oriented Messaging Protocol) jako warstwę aplikacyjną nad WebSocket. Spring udostępnia komponent `SimpMessagingTemplate`, pozwalający publikować wiadomości do logicznych kanałów (*topics*) bez konieczności zarządzania niskopoziomowymi sesjami.

Spring AOP. Programowanie aspektowe wykorzystano do realizacji modułu audytu – aspekt `AuditAspect` automatycznie rejestruje wywołania metod oznaczonych adnotacją `@Auditable`, oddzielając logikę audytu od logiki biznesowej (zob. sekcję 8).

PostgreSQL 16. Relacyjna baza danych wybrana ze względu na pełną zgodność ze standardem SQL, wsparcie dla typów `TIMESTAMP` z obsługą stref czasowych oraz dojrzałe wsparcie w Spring Data JPA.

JUnit 5 + Mockito. Standardowy stos do testów jednostkowych w ekosystemie Spring Boot. Dostępny przez zależność `spring-boot-starter-test`, która agreguje również AssertJ i Spring Test.

3.2 Frontend

React 19 + Vite 8. React stanowi obecnie standard de facto w budowie interfejsów typu SPA. Vite, w przeciwieństwie do klasycznego Webpack’a, oferuje natychmiastowy start serwera deweloperskiego i hot reload dzięki natywnemu wykorzystaniu modułów ES.

Tailwind CSS + shadcn/ui. Tailwind to biblioteka klas *utility-first*, umożliwiająca stylizację komponentów bezpośrednio w kodzie JSX. Biblioteka shadcn/ui udostępnia kolekcję dostępnych komponentów (Dialog, Form, Toast, Card itp.) zbudowanych nad Radix UI i stylizowanych Tailwindem, które można skopiować bezpośrednio do projektu.

i18next + react-i18next. Wybrane do obsługi wielojęzyczności. Tłumaczenia przechowywane są w plikach JSON (`pl.json`, `en.json`); biblioteka oferuje deklaratywny dostęp przez hook `useTranslation`.

@stomp/stompjs. Klientka implementacja protokołu STOMP nad WebSocket, w pełni kompatybilna z serwerową stroną Spring WebSocket.

axios. Klient HTTP używany do żądań REST. Zaletą nad natywnym `fetch` jest spójna obsługa nagłówków, interceptory żądań/odpowiedzi (wykorzystane do automatycznego doczepiania tokenu JWT) oraz spójne zachowanie w starszych przeglądarkach.

date-fns + sonner + next-themes. Biblioteki uzupełniające: `date-fns` – formatowanie dat z lokalizacją, `sonner` – toasty (powiadomienia), `next-themes` – przełączanie trybu jasnego/ciemnego z zapamiętaniem preferencji.

3.3 Infrastruktura

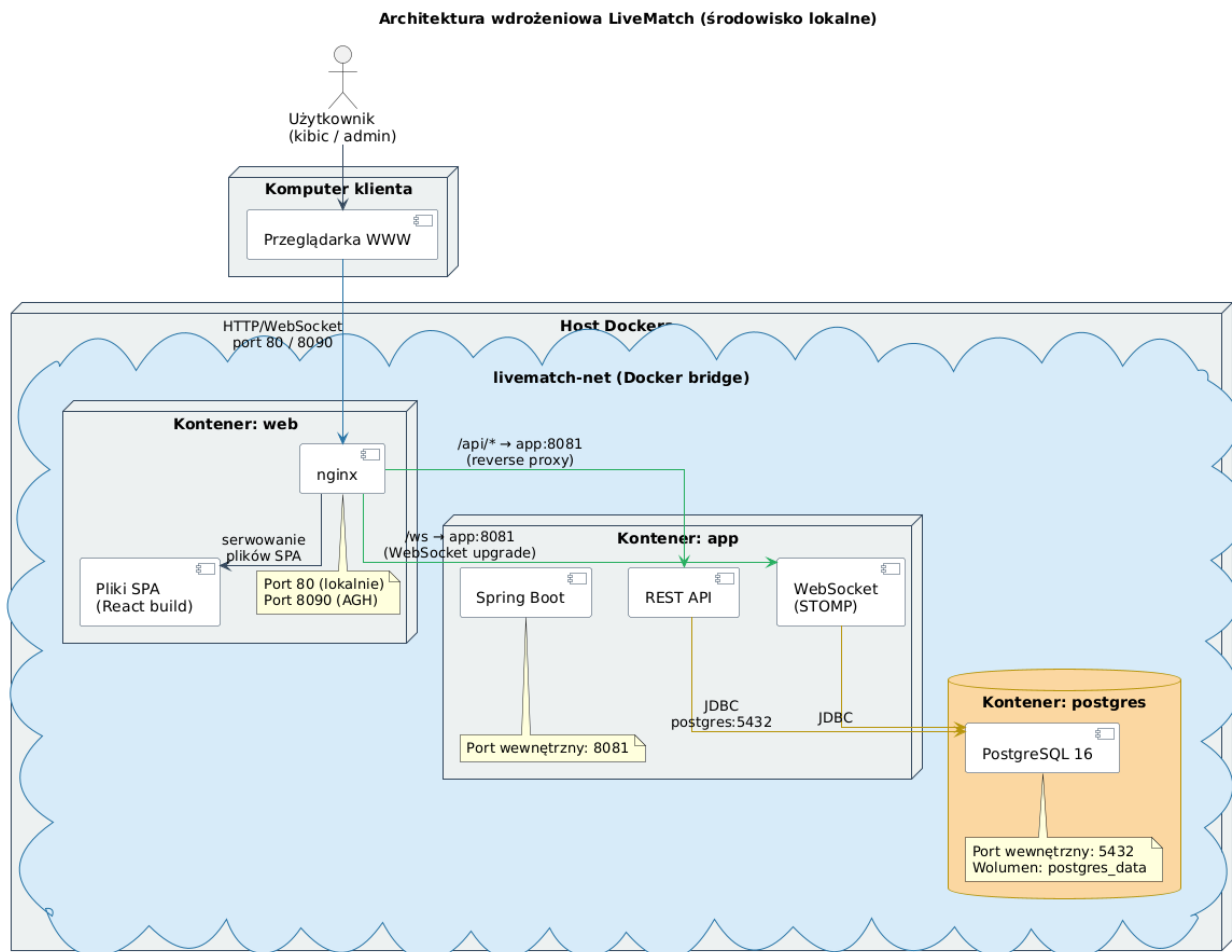
Docker + Docker Compose. Konteneryzacja zapewnia identyczność środowiska deweloperskiego, testowego i produkcyjnego. Backend, frontend i baza danych zostały umieszczone w osobnych kontenerach, łączonych przez sieć mostkową Dockera.

nginx. W kontenerze frontendu pełni dwie funkcje: serwowanie zbudowanych plików statycznych SPA oraz *reverse proxy* dla żądań kierowanych do backendu (`/api/*` oraz `/ws`). Dzięki temu z perspektywy przeglądarki frontend i backend działają na tym samym adresie, eliminując problemy CORS w produkcji.

4 Architektura systemu

4.1 Architektura wdrożeniowa

W środowisku lokalnym aplikacja składa się z trzech kontenerów Docker: `postgres`, `app` (backend) oraz `web` (frontend wraz z `nginx` jako reverse proxy). Diagram wdrożeniowy przedstawia rysunek 2.



Rysunek 2: Diagram wdrożenia: trzy kontenery Docker w jednej sieci mostkowej

Na maszynie wirtualnej AGH wykorzystywana jest baza danych PostgreSQL, hostowana na maszynie `pascal.fis.agh.edu.pl`, dlatego kontener `postgres` nie jest tam uruchamiany – pozostają tylko dwa kontenery (`app`, `web`), które łączą się z bazą po sieci.

4.2 Architektura logiczna

System został zaprojektowany w klasycznej, trójwarstwowej architekturze:

1. **Warstwa prezentacji** – aplikacja React (SPA) działająca w przeglądarce. Komunikuje się z backendem przez REST (axios) i WebSocket (STOMP).
2. **Warstwa logiki biznesowej** – aplikacja Spring Boot. Zawiera kontrolery REST, serwisy biznesowe, mappery DTO oraz konfigurację bezpieczeństwa, AOP i WebSocket.
3. **Warstwa danych** – baza PostgreSQL z dostępem przez Spring Data JPA / Hibernate.

Backend dodatkowo dzieli się wewnętrznie zgodnie z konwencją Spring Boot:

- **controller** – punkty wejścia REST (`@RestController`).
- **service** – logika biznesowa, walidacja reguł domeny, transakcje.
- **repository** – interfejsy Spring Data JPA.
- **domain** – encje (`@Entity`).
- **dto** – niemutowalne obiekty transferowe (Java *records*).
- **mapper** – konwertery domena $\leftrightarrow$ DTO.
- **security** – konfiguracja Spring Security, filtr JWT, narzędzia tokenowe.
- **aop** – adnotacja `@Auditable` i aspekt audytujący.
- **config** – konfiguracje (WebSocket, CORS, inicjalizacja danych).
- **exception** – niestandardowe wyjątki i globalny handler.

4.3 Przepływ danych

Przeglądarka komunikuje się wyłącznie z kontenerem **web** (port 80 lokalnie, 8090 na AGH). Wewnątrz tego kontenera nginx:

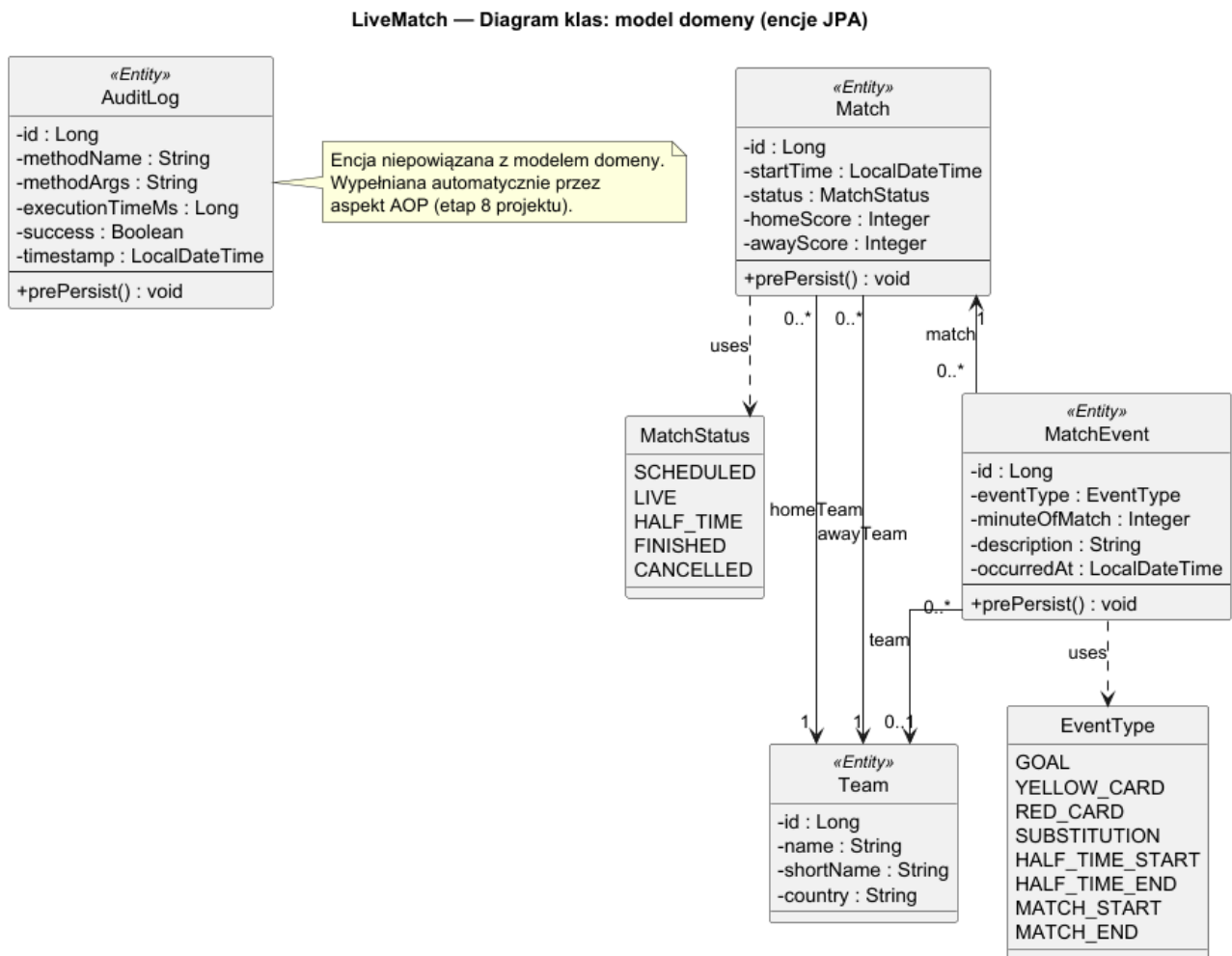
- żądania na ścieżki `/api/*` przekazuje (reverse proxy) do kontenera **app** na port 8081,
- żądania na ścieżkę `/ws` (z nagłówkami *Upgrade*) przekazuje jako WebSocket do kontenera **app**,
- wszystkie pozostałe żądania serwuje jako pliki statyczne SPA (z fallbackiem na `index.html` dla routingu po stronie klienta).

5 Model danych

5.1 Diagram klas modelu domeny

Model danych aplikacji (rys. 3) zawiera następujące encje JPA:

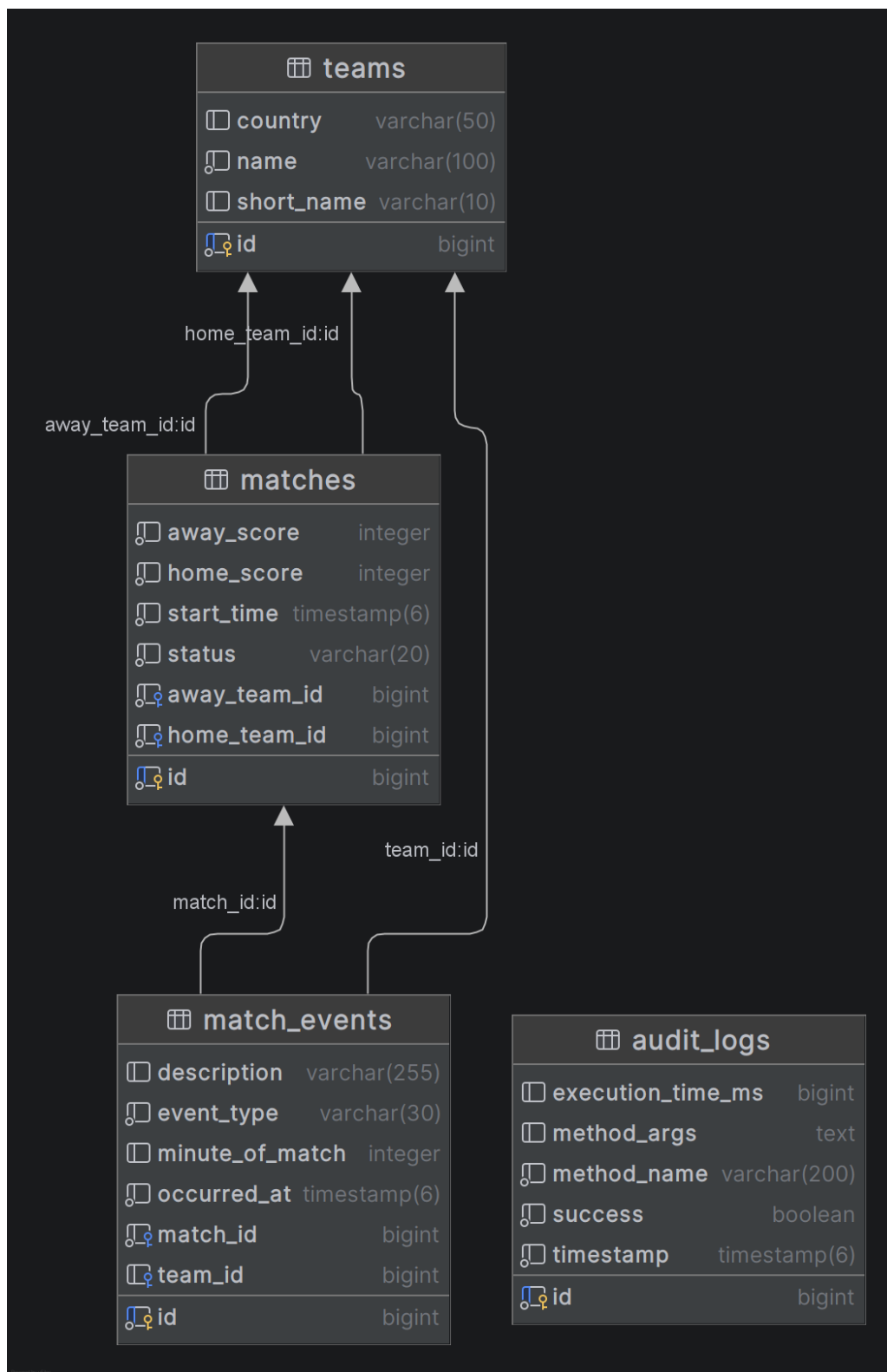
- **Team** – drużyna; może być klubem (pole `name` ustawione) lub reprezentacją narodową (`name` NULL, kraj identyfikowany kodem ISO w polu `countryCode`).
- **Match** – mecz; posiada dwie relacje @ManyToMany do **Team** (gospodarz, gość), status (**MatchStatus**), datę rozpoczęcia.
- **MatchEvent** – zdarzenie boiskowe; powiązane z meczem oraz opcjonalnie z drużyną, której dotyczy.
- **MatchStatus**, **EventType**, **Role** – typy wyliczeniowe.
- **User** – konto użytkownika systemu (administratora).
- **AuditLog** – wpis audytu, uzupełniany automatycznie przez aspekt (zob. sekcję 8).



Rysunek 3: Diagram klas modelu domeny

5.2 Diagram ERD

Fizyczna struktura bazy PostgreSQL (rys. 4) została wygenerowana automatycznie przez Hibernate na podstawie deklaracji encji. Klucze obce w tabeli `matches` (`home_team_id`, `away_team_id`) odwołują się do `teams`; tabela `match_events` referencjonuje zarówno mecz, jak i opcjonalnie drużynę.



Rysunek 4: Diagram ERD bazy danych aplikacji

5.3 Inicjalizacja danych

Po pierwszym uruchomieniu aplikacji baza jest automatycznie wypełniana danymi testowymi – 26 klubami i 48 reprezentacjami narodowymi oraz 96 meczami fazy grupowej Mistrzostw Świata 2026. Dane pochodzą z pliku `src/main/resources/data.sql`, wykonywanego przez Spring po utworzeniu schematu przez Hibernate (właściwość `spring.jpa.defer-datasource-initialization=true`). Każda instrukcja `INSERT` jest opakowana w klauzulę `WHERE NOT EXISTS`, dzięki czemu plik jest idempotentny i bezpieczny w wielokrotnym wykonaniu.

Konto administratora (`admin / admin123`) tworzone jest przez klasę `DataInitializer` implementującą interfejs `CommandLineRunner`, uruchamianą po pełnej inicjalizacji kontekstu Springa.

6 Warstwa serwerowa

6.1 Kontrolery REST

Aplikacja wystawia cztery kontrolery REST, każdy odpowiedzialny za jeden zasób:

Kontroler	Bazowa ścieżka	Operacje
AuthController	/api/auth	POST /login
TeamController	/api/teams	GET, POST, PUT, DELETE
MatchController	/api/matches	GET, POST, PATCH /status, DELETE
MatchEventController	/api/matches/{id}/events	GET, POST

Wszystkie kontrolery zwracają obiekty typu `record` z pakietu `dto`, nigdy bezpośrednio encji JPA – co eliminuje ryzyko leniwego ładowania poza kontekstem transakcji oraz pozwala stabilizować API niezależnie od zmian w modelu domeny.

6.2 Warstwa serwisów

Serwisy zawierają właściwą logikę biznesową, walidację reguł domeny oraz zarządzanie transakcjami. Listing 1 pokazuje fragment klasy `MatchService` odpowiedzialnej za tworzenie meczu – z walidacją reguły biznesowej (drużyna nie może grać sama ze sobą) oraz natychmiastowym opublikowaniem zdarzenia na kanale `WebSocket`.

```
1 @Auditable
2 public MatchDto create(MatchCreateDto dto) {
3     if (dto.homeTeamId().equals(dto.awayTeamId())) {
4         throw new BusinessException(
5             "A team cannot play against itself");
6     }
7     Match match = Match.builder()
8         .homeTeam(teamService.findOrThrow(dto.homeTeamId()))
9         .awayTeam(teamService.findOrThrow(dto.awayTeamId()))
10        .startTime(dto.startTime())
11        .build();
12
13    MatchDto result = matchMapper.toDto(matchRepository.save(match));
14
15    messagingTemplate.convertAndSend("/topic/matches/created", result);
16
17    return result;
18 }
```

Listing 1: Fragment `MatchService` – tworzenie meczu

Adnotacja `@Auditable` (linia 1) sygnalizuje, że wywołanie tej metody zostanie automatycznie zarejestrowane przez aspekt `AuditAspect`, bez konieczności umieszczania w niej kodu audytu (zob. sekcję 8).

6.3 Repozytoria Spring Data JPA

Wszystkie repozytoria zdefiniowane są jako interfejsy rozszerzające `JpaRepository<T, Long>`. Spring Data automatycznie generuje implementację zapytań CRUD oraz zapytań pochodnych od nazwy metody (np. `findByStatus`, `existsByUsername`). Dla zapytań nietrywialnych wykorzystywana jest adnotacja `@Query` z JPQL.

6.4 Walidacja i obsługa wyjątków

Walidacja danych wejściowych jest realizowana adnotacjami `jakarta.validation` (`@NotNull`, `@NotBlank`, `@Size`, `@Future`) bezpośrednio na rekordach DTO. Spring automatycznie waliduje obiekt przy adnotacji `@Valid` w sygnaturze metody kontrolera.

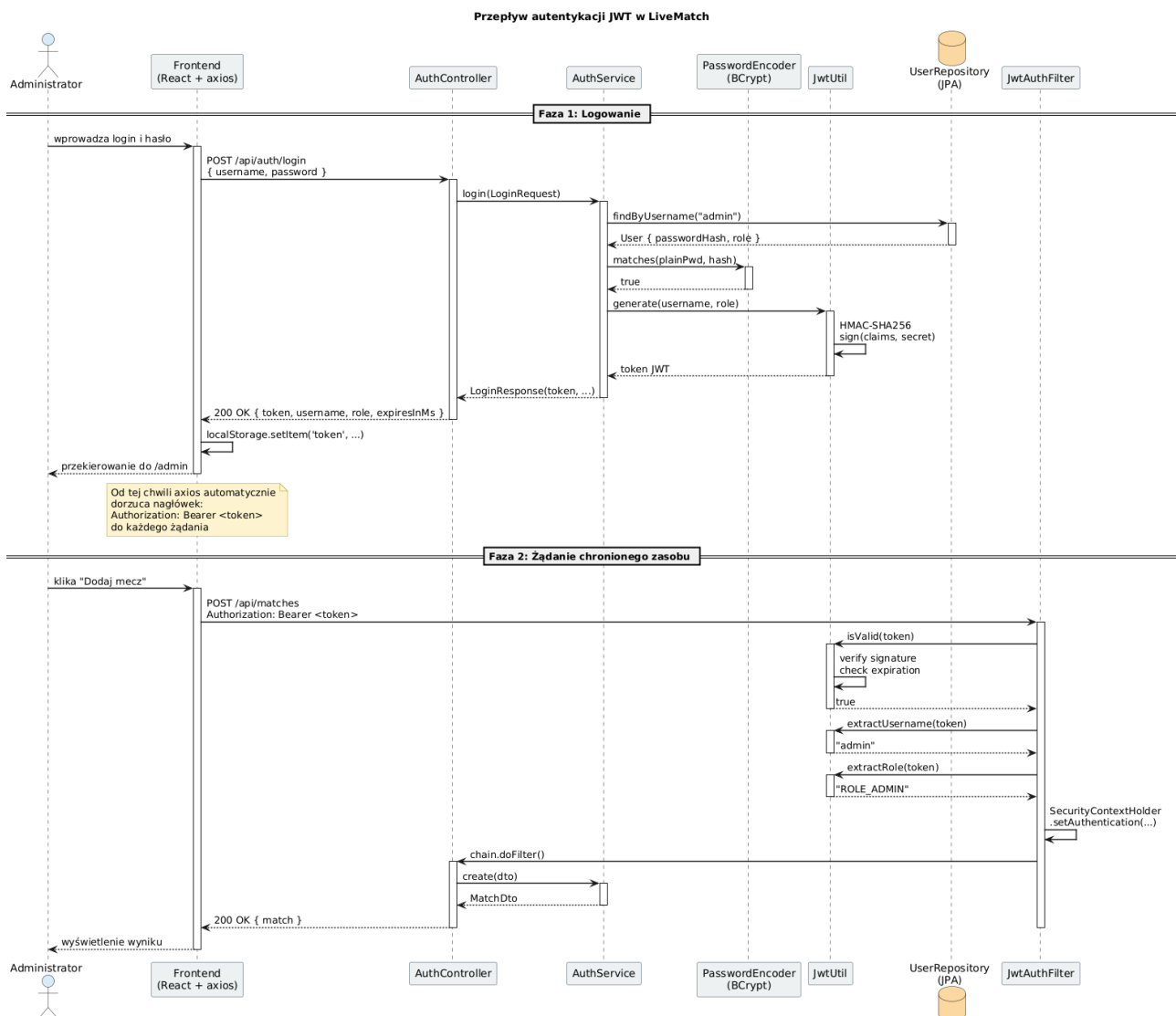
Wszystkie wyjątki są przechwytywane przez `GlobalExceptionHandler` (`@RestControllerAdvice`) i tłumaczone na spójne odpowiedzi JSON z kodem HTTP, wiadomością i polem `timestamp`. Niestandardowe wyjątki domeny (`ResourceNotFoundException`, `BusinessRuleViolationException`) są mapowane odpowiednio na 404 i 400.

7 Bezpieczeństwo

7.1 Architektura JWT

System wykorzystuje tokeny JSON Web Token w trybie *stateless*: serwer nie przechowuje sesji, każde żądanie jest niezależnie weryfikowane na podstawie tokenu w nagłówku **Authorization**. Schemat działania (rys. 5) jest następujący:

1. Klient wysyła żądanie **POST /api/auth/login** z parą login + hasło.
2. **AuthService** weryfikuje hasło algorytmem **BCrypt**; w przypadku zgodności generuje token JWT podpisany kluczem **HMAC SHA-256**.
3. Klient zapisuje token (w **localStorage**) i dołącza go do każdego kolejnego żądania w nagłówku **Authorization: Bearer <token>**.
4. Filtr **JwtAuthFilter** (zaimplementowany jako **OncePerRequestFilter**) na każde żądanie sprawdza nagłówek, weryfikuje podpis i datę wygaśnięcia tokenu, po czym ustawia obiekt **Authentication** w **SecurityContextHolder**.



Rysunek 5: Przeływ autentykacji JWT

Klasa `JwtUtil` odpowiada za generowanie i weryfikację tokenów. Wykorzystuje bibliotekę `JJWT 0.12.6`, klucz tajny jest wczytywany z konfiguracji (`app.jwt.secret`, `BASE64`), a domyślny czas życia tokenu wynosi 24 godziny.

7.2 Konfiguracja Spring Security

Konfiguracja `SecurityFilterChain` (listing 2) definiuje zasady dostępu do poszczególnych ścieżek REST. Endpointy autoryzacji (`/api/auth/**`), `health-check` oraz `handshake WebSocket` są publiczne; operacje odczytu na meczach i drużynach (`GET`) również nie wymagają logowania (dostęp dla kibica). Wszystkie operacje modyfikujące (`POST`, `PUT`, `PATCH`, `DELETE`) wymagają roli `ROLE_ADMIN`.

```
1 http
2   .csrf(AbstractHttpConfigurer::disable)
3   .cors(c -> {})
4   .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy
5     .STATELESS))
6   .authorizeHttpRequests(auth -> auth
7     .requestMatchers("/api/auth/**").permitAll()
8     .requestMatchers("/actuator/health").permitAll()
9     .requestMatchers("/ws/**").permitAll()
10    .requestMatchers(HttpMethod.GET, "/api/teams/**").permitAll()
11    .requestMatchers(HttpMethod.GET, "/api/matches/**").permitAll()
12    .requestMatchers(HttpMethod.POST, "/api/**").hasRole("ADMIN")
13    .requestMatchers(HttpMethod.PUT, "/api/**").hasRole("ADMIN")
14    .requestMatchers(HttpMethod.PATCH, "/api/**").hasRole("ADMIN")
15    .requestMatchers(HttpMethod.DELETE, "/api/**").hasRole("ADMIN")
16    .anyRequest().authenticated())
    .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.
      class);
```

Listing 2: Konfiguracja Spring Security

Ochrona CSRF została wyłączona, ponieważ aplikacja jest stateless (brak sesji ciasteczkowych), a tokeny JWT same w sobie chronią przed atakami CSRF (nie są automatycznie wysyłane przez przeglądarkę).

7.3 Hashowanie haseł

Hasła użytkowników są hashowane algorytmem `BCrypt` (z domyślnym *cost factor* 10), zapewniającym odporność na ataki słownikowe dzięki wbudowanej soli i celowemu spowolnieniu funkcji haszującej. Bean `PasswordEncoder` jest deklarowany w `SecurityConfig` jako instancja `BCryptPasswordEncoder`.

8 Programowanie aspektowe

8.1 Idea

Programowanie aspektowe (AOP) służy do modularyzacji zagadnień przekrojowych (*cross-cutting concerns*) – takich jak logowanie, audyt, monitorowanie wydajności czy zarządzanie transakcjami – które naturalnie przeplatałyby się przez logikę biznesową aplikacji. W projekcie *LiveMatch* AOP wykorzystano do realizacji modułu audytu kluczowych operacji.

8.2 Adnotacja @Auditable

Mechanizm bazuje na własnej adnotacji `@Auditable`, którą można nałożyć na dowolną metodę warstwy serwisowej. Adnotacja pełni rolę *markera* – nie wykonuje żadnej logiki sama z siebie, lecz sygnalizuje aspektowi, że dana metoda podlega audytowi.

Przykład zastosowania widoczny jest w listingu 1 (linia 1): metoda `create` klasy `MatchService` jest oznaczona `@Auditable`, dzięki czemu każde jej wywołanie zostanie zarejestrowane.

8.3 Aspekt AuditAspect

Aspekt jest zaimplementowany jako klasa oznaczona adnotacją `@Aspect` i `@Component`. Zawiera *advice* typu *around*, który:

1. Mierzy czas wykonania metody (`System.currentTimeMillis()`).
2. Wywołuje oryginalną metodę przez `proceedingJoinPoint.proceed()`.
3. Rejestruje wpis w tabeli `audit_logs` zawierający: nazwę klasy i metody, argumenty, czas wykonania, znacznik czasowy oraz nazwę użytkownika (z `SecurityContextHolder`).

Mechanizm działa w pełni transparentnie dla wywołującego – z perspektywy klienta nie ma różnicy między wywołaniem metody oznaczonej `@Auditable` a zwykłej. Spring AOP korzysta z dynamicznych proxy CGLIB do owijania komponentów Springa.

8.4 Encja AuditLog

Tabela `audit_logs` jest strukturalnie niezależna od pozostałych tabel domeny – nie posiada kluczy obcych. Pola zawierają informacje pozwalające na późniejszą analizę aktywności systemu, w tym potencjalne odtworzenie ciągu operacji administracyjnych przed wystąpieniem incydentu.

8.5 Schemat działania

Dynamikę wpięcia aspektu w wywołanie metody przedstawia diagram sekwencji (rys. 6, omówiony w sekcji 9). Zwracają uwagę dwa kluczowe momenty:

- kontroler `MatchEventController` wywołuje metodę serwisu, ale w rzeczywistości wywołanie trafia najpierw do dynamicznego proxy generowanego przez Springa,
- proxy uruchamia `AuditAspect.around()`, który wykonuje oryginalną metodę i następnie zapisuje wpis audytu.

9 Komunikacja w czasie rzeczywistym

9.1 WebSocket i STOMP

Komunikacja asynchroniczna serwer → klient została zrealizowana w oparciu o WebSocket z protokołem warstwy aplikacyjnej STOMP (*Simple Text Oriented Messaging Protocol*). STOMP wprowadza koncepcję *topics* (kanałów logicznych), do których klienci mogą subskrybować się, a serwer publikować wiadomości – nie wymaga to ręcznego śledzenia sesji klientów.

9.2 Konfiguracja serwera

Konfiguracja WebSocket (klasa `WebSocketConfig`) deklaruje endpoint handshake'u (`/ws`), prefiks dla kanałów (`/topic`) oraz prefiks dla wiadomości kierowanych przez klienta do serwera (`/app`).

```
1 @Override
2 public void configureMessageBroker(MessageBrokerRegistry config) {
3     config.enableSimpleBroker("/topic");
4     config.setApplicationDestinationPrefixes("/app");
5 }
6
7 @Override
8 public void registerStompEndpoints(StompEndpointRegistry registry) {
9     registry.addEndpoint("/ws")
10        .setAllowedOriginPatterns("http://localhost:*", "http://127.0.0.1:*");
11 }
```

Listing 3: `WebSocketConfig` – konfiguracja STOMP

9.3 Kanały tematyczne

Aplikacja wykorzystuje następujące kanały:

Kanał	Treść wiadomości
<code>/topic/matches/created</code>	nowo utworzony mecz (DTO)
<code>/topic/matches/{id}/status</code>	zmiana statusu meczu (np. start, koniec)
<code>/topic/matches/{id}/events</code>	nowe zdarzenie boiskowe (gol, kartka, ...)

9.4 Publikacja zdarzeń

Po stronie serwera publikacja realizowana jest przez `SimpMessagingTemplate`:

```
1 messagingTemplate.convertAndSend("/topic/matches/" + matchId + "/events",
    eventDto);
```

9.5 Subskrypcja po stronie klienta

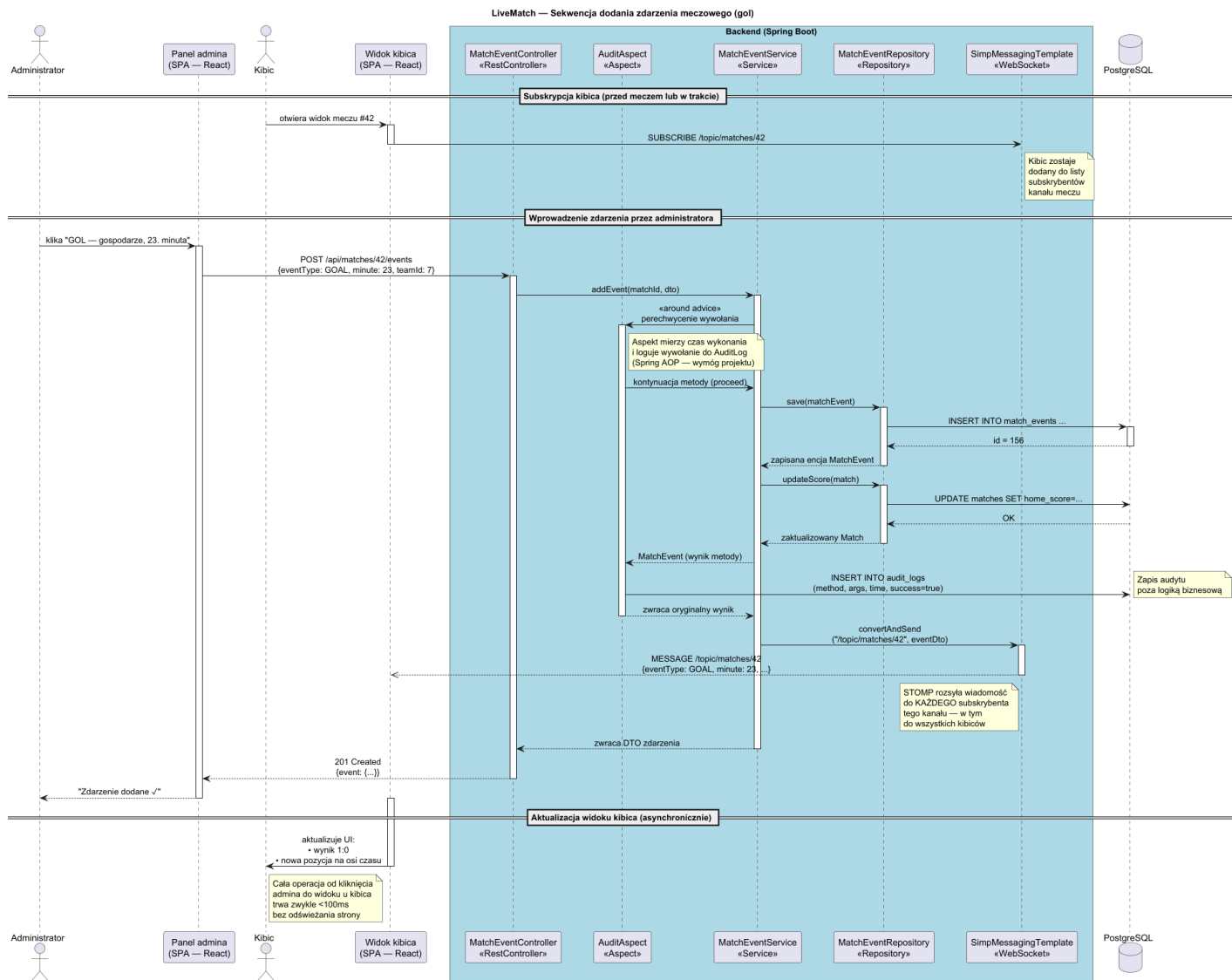
Po stronie klienta wykorzystywana jest biblioteka `@stomp/stompjs`. Logika subskrypcji została wydzielona do reużywalnego React hooka `useMatchUpdates` (listing 4), który automatycznie obsługuje (re)połączenie i sprząta po rozmontowaniu komponentu.

```
1 export function useMatchUpdates(matchId, { onEvent, onStatusChange }) {
2   useEffect(() => {
3     const wsUrl = import.meta.env.VITE_WS_URL
4       || 'ws://${window.location.host}/ws';
5
6     const client = new Client({
7       brokerURL: wsUrl,
8       reconnectDelay: 5000,
9       onConnect: () => {
10        client.subscribe(`/topic/matches/${matchId}/events`, (
11          message) => {
12            onEvent?.(JSON.parse(message.body));
13          });
14        client.subscribe(`/topic/matches/${matchId}/status`, (
15          message) => {
16            onStatusChange?.(JSON.parse(message.body));
17          });
18      },
19    });
20    client.activate();
21    return () => client.deactivate();
22  }, [matchId, onEvent, onStatusChange]);
23 }
```

Listing 4: Hook `useMatchUpdates` - subskrypcja STOMP

9.6 Diagram sekwencji – dodanie zdarzenia

Pełen scenariusz rejestracji zdarzenia przez administratora i jego propagacji do kibica obserwującego mecz przedstawia diagram sekwencji (rys. 6). Diagram obejmuje trzy fazy: wcześniejszą subskrypcję kanału meczu, obsługę żądania REST od panelu administratora oraz asynchroniczną aktualizację UI po stronie kibica.



Rysunek 6: Diagram sekwencji: dodanie zdarzenia meczowego i propagacja do klientów

10 Frontend

10.1 Routing i kontrola dostępu

Aplikacja kliencka wykorzystuje bibliotekę `react-router-dom` (wersja 7). Ścieżki publiczne (`/`, `/matches/:id`, `/login`) są dostępne bez logowania; ścieżki `/admin/*` są chronione przez komponent `ProtectedRoute`, który sprawdza obecność i ważność tokenu w `AuthContext` oraz w razie potrzeby przekierowuje do strony logowania.

10.2 Zarządzanie stanem

Globalny stan aplikacji ograniczony jest do dwóch kontekstów React:

- `AuthContext` – przechowuje token JWT, dane zalogowanego użytkownika i funkcje `login/logout`. Dodatkowo realizuje automatyczne wylogowanie po wygaśnięciu tokenu (parsowanie pola `exp` JWT i ustawienie `setTimeout`).
- `ThemeContext` (z biblioteki `next-themes`) – zarządza preferencją trybu jasnego/ciemnego.

Stan lokalny komponentów obsługiwany jest standardowymi hookami React (`useState`, `useEffect`, `useMemo`).

10.3 Komunikacja z API

Warstwa serwisowa (`services/`) wykorzystuje instancję `axios` z dwoma interceptorami:

- *request interceptor* – automatycznie doczepia nagłówek `Authorization: Bearer <token>` jeżeli token istnieje w `localStorage`,
- *response interceptor* – przechwytuje odpowiedzi 401, czyści `localStorage` i przekierowuje na stronę logowania.

10.4 Internacjonalizacja

Tłumaczenia są przechowywane w plikach JSON w katalogu `src/locales/` (`pl.json`, `en.json`) i organizowane hierarchicznie w sekcje (`common`, `auth`, `match`, `admin`, `country`, ...). Każdy komponent uzyskuje dostęp do tłumaczeń przez hook `useTranslation`:

```
1 const { t } = useTranslation();  
2 return <h1>{t("match.list.title")}</h1>;
```

Listing 5: Wykorzystanie `i18next` w komponencie

Wybór języka jest zapamiętywany w `localStorage` dzięki konfiguracji modułu `i18next-browser-languagedetector`. Daty są dodatkowo formatowane biblioteką `date-fns` z odpowiednim `locale` (`pl/en-US`).

10.5 Komponenty UI

Warstwa wizualna opiera się na komponentach z biblioteki `shadcn/ui` (`Dialog`, `Card`, `Button`, `Input`, `Select`, `Table`, `Toast`), stylizowanych klasami Tailwind CSS. Wszystkie komponenty są w pełni responsywne i wspierają tryb ciemny dzięki klasie `dark`: w klasach utility.

11 Konteneryzacja i wdrożenie

11.1 Multi-stage build backendu

Plik `Dockerfile` w głównym katalogu projektu definiuje dwustopniowy build backendu. W pierwszym etapie używany jest obraz `maven:3.9-eclipse-temurin-21`, w którym kod źródłowy jest kompilowany i pakowany do JAR-a. W drugim etapie wynikowy JAR jest kopiowany do znacznie lżejszego obrazu `eclipse-temurin:21-jre`, dzięki czemu finalny obraz Dockera nie zawiera Mavena ani plików źródłowych. Aplikacja działa pod nieprzywilegowanym użytkownikiem `spring`, z konfiguracją JVM-a optymalizowaną dla kontenerów (`-XX:MaxRAMPercentage=75.0 -XX:+UseG1GC`).

11.2 Multi-stage build frontendu

Analogicznie, plik `client/Dockerfile` buduje frontend w dwóch etapach. Etap pierwszy (`node:20-alpine`) instaluje zależności i wykonuje `npm run build`, generując zoptymalizowane pliki statyczne w katalogu `dist/`. Etap drugi (`nginx:alpine`) kopiuje pliki statyczne wraz z niestandardową konfiguracją `nginx.conf` do obrazu produkcyjnego.

11.3 Konfiguracja nginx

Plik `client/nginx.conf` pełni dwie funkcje: serwowanie SPA z fallbackiem na `index.html` (wymagane dla routingu po stronie klienta) oraz reverse proxy dla żądań REST i WebSocket:

```
1 location /api/ {
2     proxy_pass http://app:8081/api/;
3     proxy_set_header Host $host;
4     proxy_set_header X-Real-IP $remote_addr;
5 }
6
7 location /ws {
8     proxy_pass http://app:8081/ws;
9     proxy_http_version 1.1;
10    proxy_set_header Upgrade $http_upgrade;
11    proxy_set_header Connection "upgrade";
12 }
13
14 location / {
15     try_files $uri $uri/ /index.html;
16 }
```

Listing 6: Fragment `nginx.conf` – reverse proxy

11.4 Kompozycja Docker Compose

Plik `docker-compose.yml` w repozytorium jest skonfigurowany pod wdrożenie na maszynie wirtualnej AGH. Definiuje dwa serwisy: `app` (backend Spring Boot) oraz `web` (frontend wraz z `nginx`), połączone wspólną siecią mostkową `livematch-net`. Wartości zmiennych `SPRING_DATASOURCE_*` wskazują na zewnętrzny serwer bazy danych AGH (`pascal.fis.agh.edu.pl`), gdzie każdy student grupy ZTI posiada własną instancję bazy o nazwie odpowiadającej loginowi AGH. Frontend wystawiony jest na port **8090**, ponieważ port 80 na maszynie AGH zajmuje współdzielony kontener `zti01_haproxy_1` grupy.

W trakcie developmentu lokalnego stosowana była rozszerzona kompozycja z dodatkowym kontenerem `postgres` (z wolumenem zapewniającym trwałość danych i `healthcheck`'iem warunkującym start backendu) oraz frontem wystawionym na port 80. Ostatecznie w repozytorium pozostawiono jedynie wariant wdrożeniowy, ponieważ stanowi on punkt końcowy projektu.

11.5 Diagram wdrożeniowy

Architekturę wdrożeniową przedstawia rysunek 2. Oba kontenery działają w jednej sieci Dockera, izolowanej od reszty systemu hosta; jedynym wystawionym na zewnątrz portem jest 8090 (frontend). Komunikacja z bazą danych odbywa się po zewnętrznej sieci AGH.

12 Testy

W ramach projektu zaimplementowano zestaw **20 testów** backendowych pokrywających kluczowe ścieżki logiki biznesowej. Testy są uruchamiane standardowym poleceniem `mvn test` i wykorzystują framework JUnit 5 oraz Mockito do mockowania zależności.

12.1 Wybrane testy jednostkowe

MatchServiceTest – testy operacji na meczach:

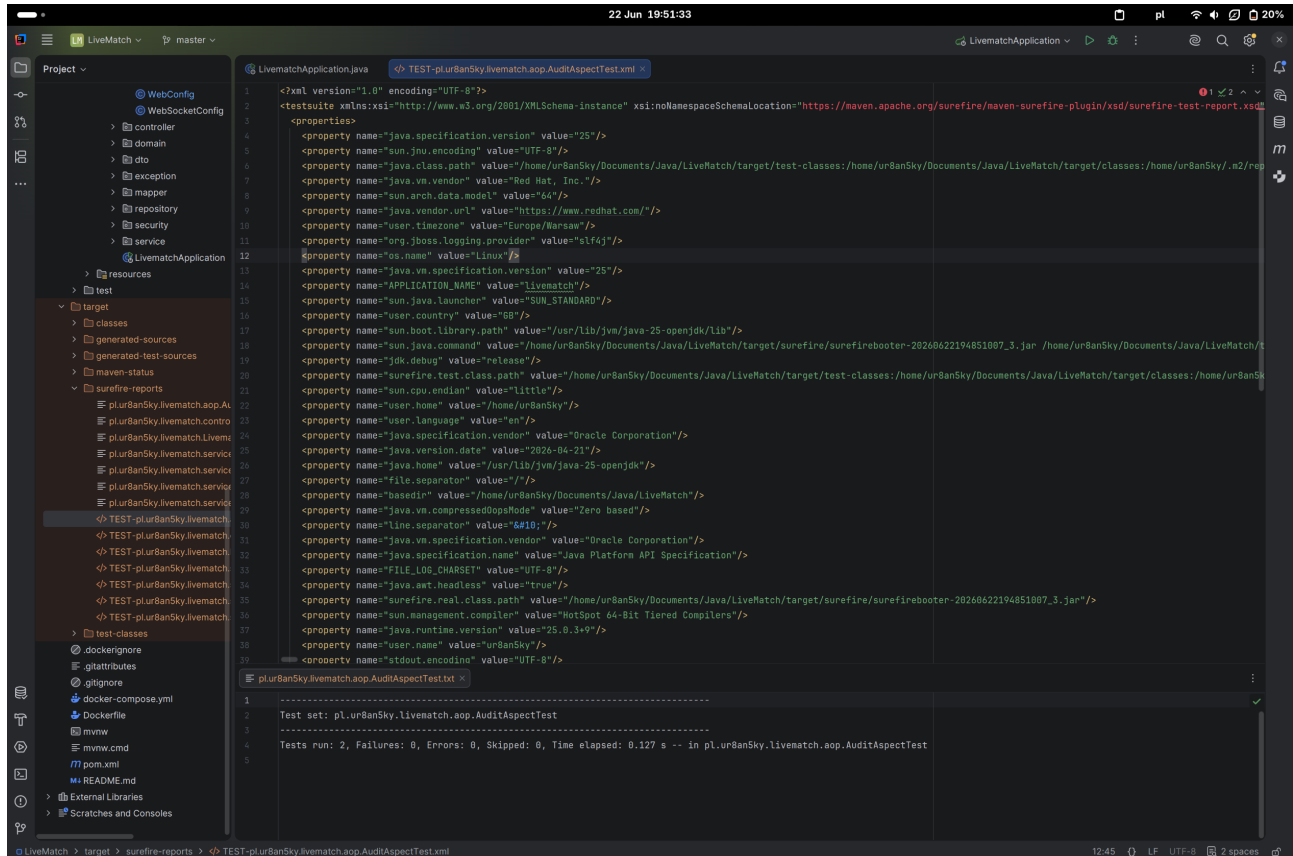
- `shouldCreateMatchWhenDataValid` – tworzenie meczu z poprawnymi danymi.
- `shouldThrowWhenHomeTeamEqualsAwayTeam` – walidacja zakazu meczu z samą sobą.
- `shouldThrowWhenTeamNotFound` – propagacja `ResourceNotFoundException`.
- `shouldRejectStatusTransitionFromFinished` – walidacja przejść statusu.

MatchEventServiceTest – testy zdarzeń boiskowych:

- `shouldAddEventToOngoingMatch` – dodanie zdarzenia do trwającego meczu.
- `shouldRejectEventOnScheduledMatch` – walidacja: zdarzenia tylko dla statusu LIVE.
- `shouldPublishEventToWebSocketChannel` – weryfikacja publikacji STOMP.

12.2 Generowanie raportu

Po wykonaniu `mvn test` szczegółowy raport dostępny jest w katalogu `target/surefire-reports/`. Wyciąg z raportu testów dla wybranej klasy serwisowej przedstawia rys. 7.

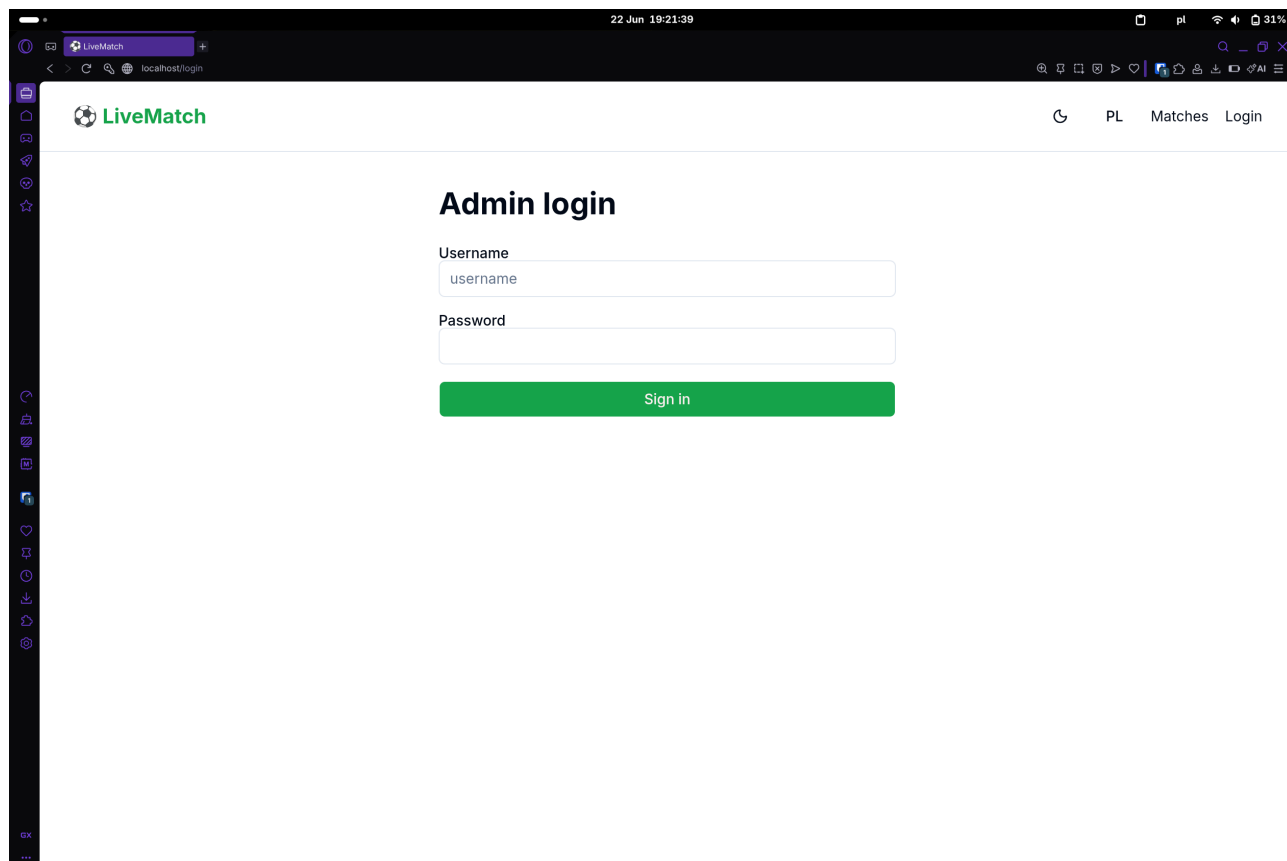


Rysunek 7: Wyciąg z raportu testów jednostkowych

13 Podręcznik użytkownika

13.1 Logowanie

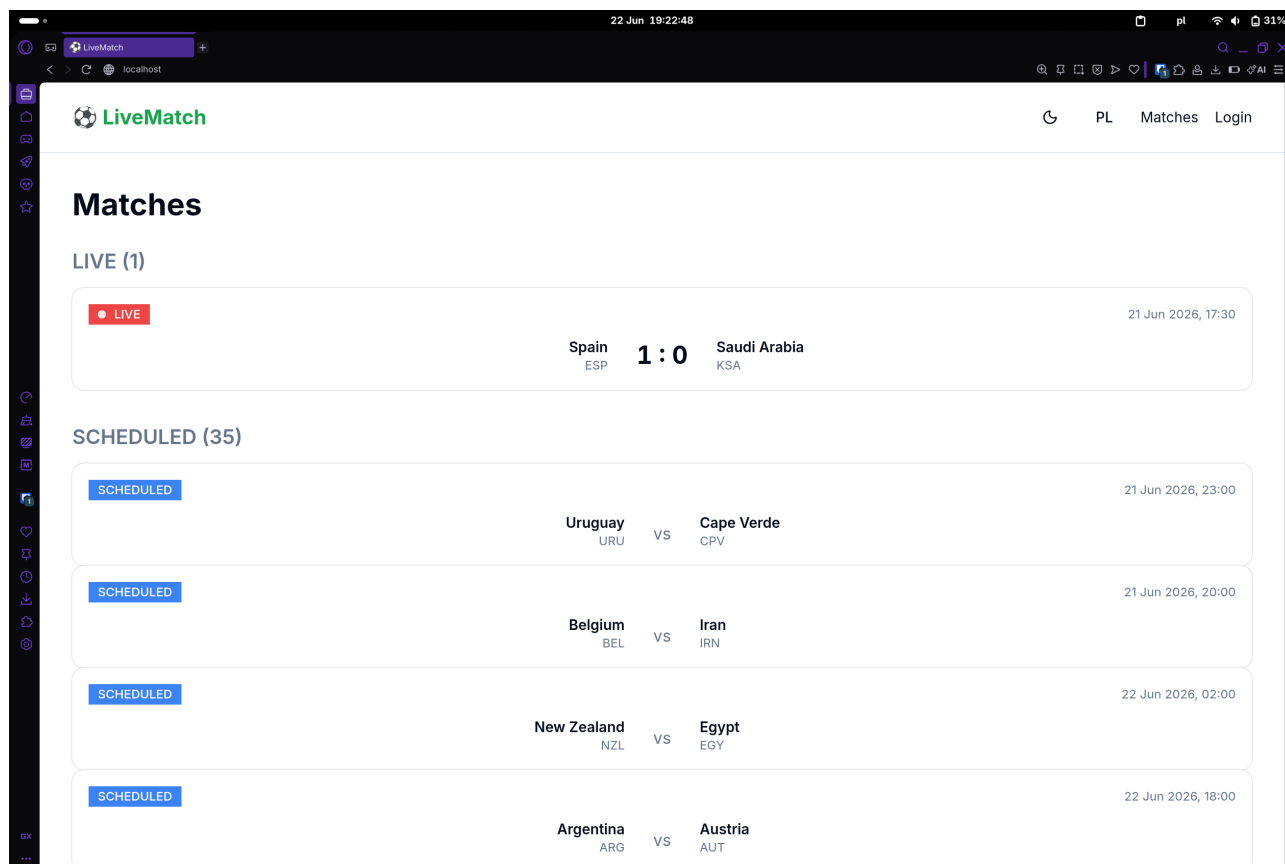
Strona logowania (rys. 8) dostępna jest pod adresem `/login`. Po wprowadzeniu loginu i hasła administrator zostaje przekierowany do panelu zarządzania. Domyślne dane to `admin / admin123`.



Rysunek 8: Strona logowania administratora

13.2 Widok kibica – lista meczów

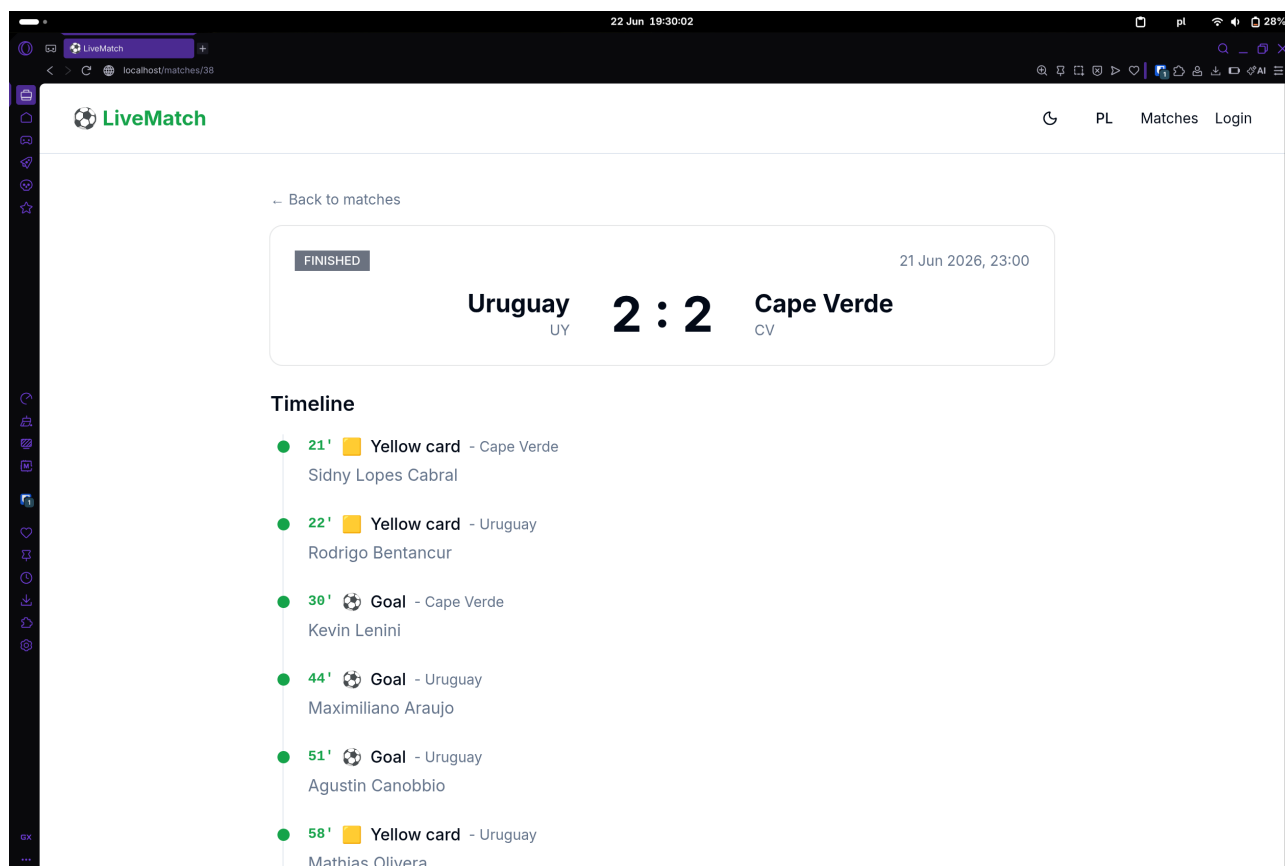
Strona główna (rys. 9) wyświetla listę meczów z możliwością filtrowania według statusu. Każdy mecz reprezentowany jest jako karta z flagami drużyn, datą rozpoczęcia, aktualnym wynikiem i statusem. Mecze trwające (*LIVE*) wyróżnione są kolorystycznie.



Rysunek 9: Lista meczów – widok kibica

13.3 Widok kibica – szczegóły meczu

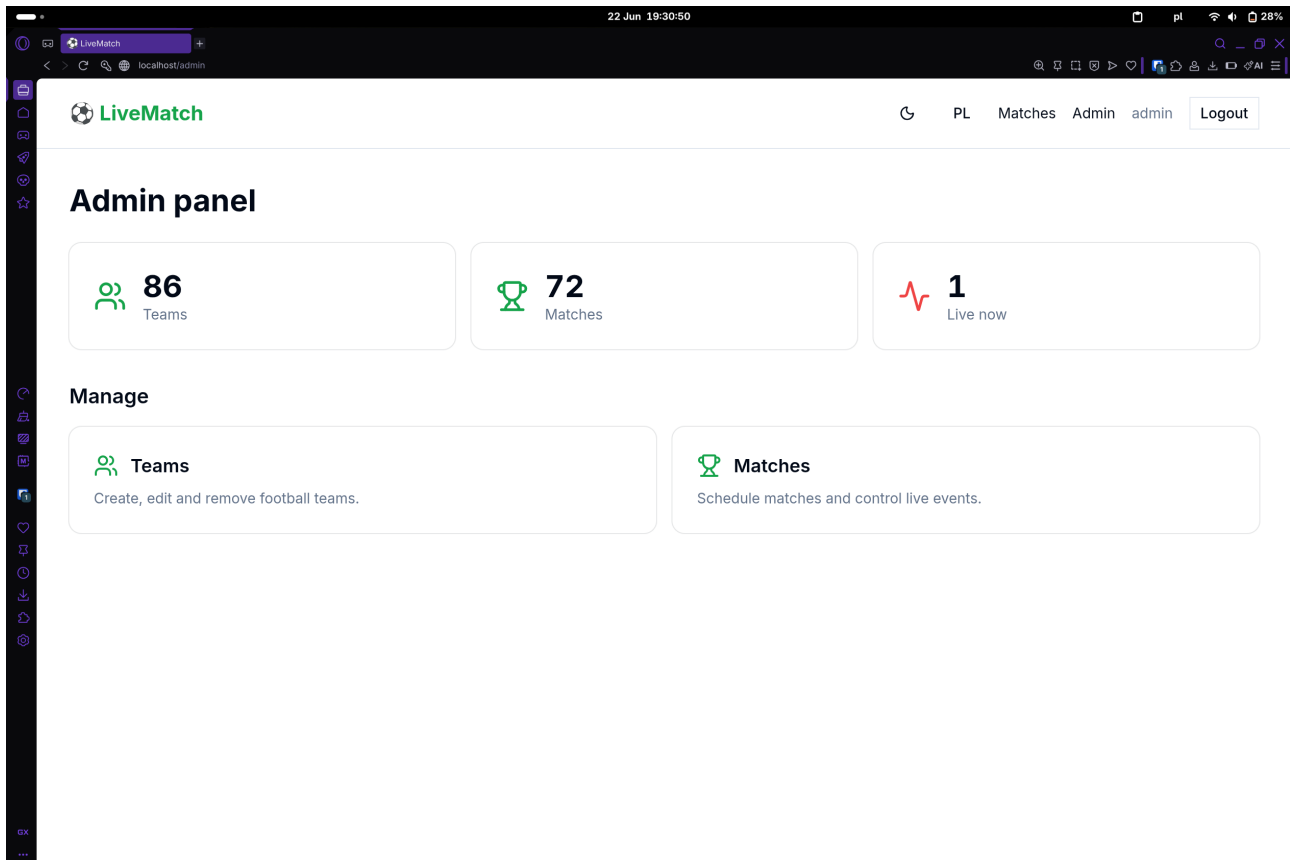
Po kliknięciu w wybrany mecz użytkownik trafia na stronę szczegółów (rys. 10), prezentującą drużyny, wynik, oś czasu zdarzeń oraz status meczu. Dla meczów trwających strona jest aktualizowana w czasie rzeczywistym dzięki subskrypcji kanału WebSocket.



Rysunek 10: Szczegóły meczu z osią czasu zdarzeń

13.4 Panel administratora

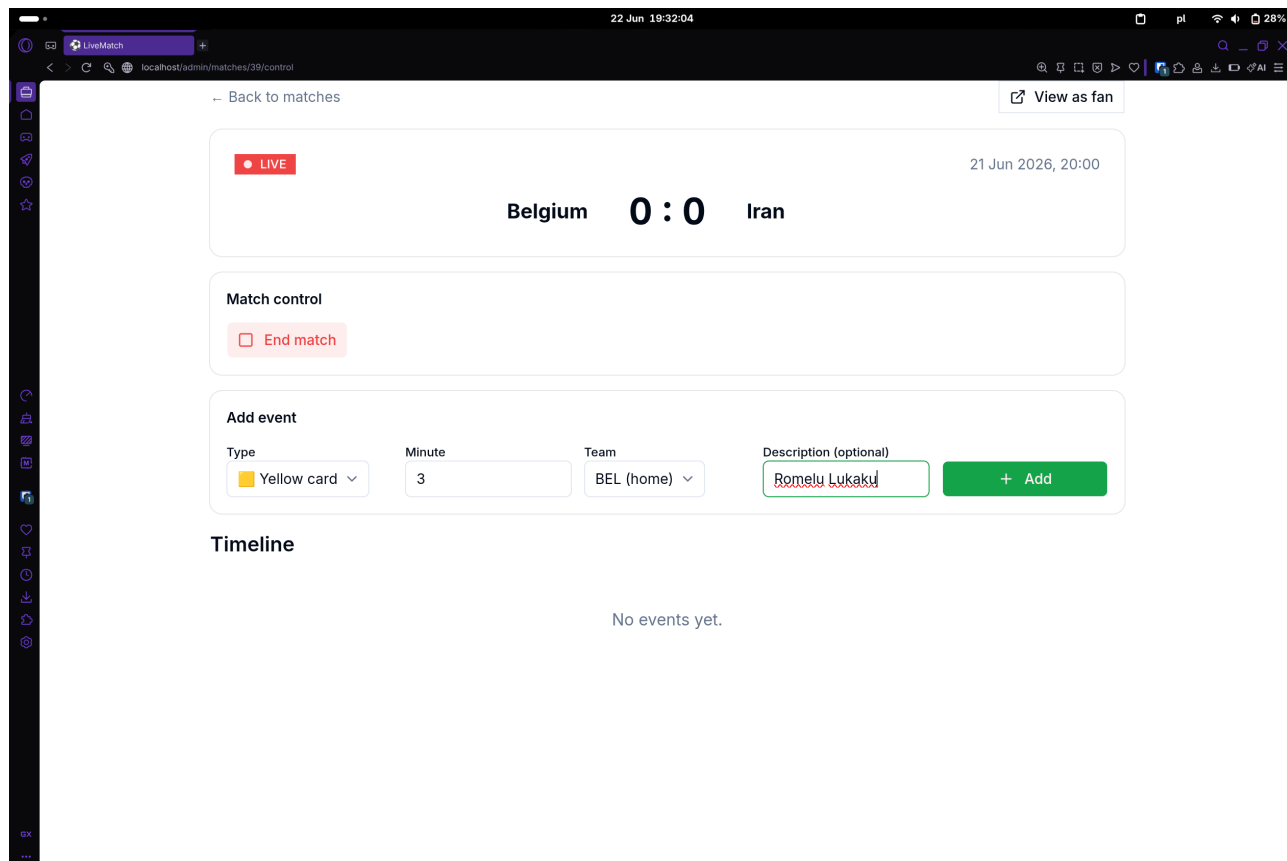
Panel administracyjny (rys. 11) udostępnia zarządzanie drużynami, meczami oraz sterowanie trwającym meczem. Dostępny po zalogowaniu pod adresem `/admin`.



Rysunek 11: Główny widok panelu administratora

13.5 Sterowanie meczem na żywo

Najbardziej dynamiczny widok aplikacji (rys. 12) umożliwia administratorowi zarządzanie meczem w trakcie jego trwania: zmianę statusu (start, koniec połowy, koniec meczu) oraz dodawanie zdarzeń boiskowych. Każda zmiana jest natychmiast widoczna u wszystkich kibiców subskrybujących mecz.



Rysunek 12: Panel sterowania trwającym meczem

13.6 Dodawanie zdarzenia

Formularz dodawania zdarzenia (rys. 13) pozwala wybrać typ zdarzenia, drużynę, której dotyczy, oraz minutę meczu. Walidacja po stronie klienta i serwera zapewnia spójność danych.

The screenshot displays a web application interface for managing matches. At the top, there's a 'Back to admin' link and a '+ New match' button. Below this is a table titled 'Matches' with columns: Status, Match, Score, Start time, and Actions. The table lists several matches, including 'Spain vs Saudi Arabia' (LIVE, 1:0), 'Belgium vs Iran' (LIVE, 0:0), and several 'SCHEDULED' matches. A modal form titled 'New match' is open in the center, allowing users to add a new match. The form includes dropdown menus for 'Home team' and 'Away team', a date/time picker for 'Start time', and 'Cancel' and 'Create match' buttons.

Status	Match	Score	Start time	Actions
LIVE	Spain vs Saudi Arabia	1 : 0	21 Jun 2026, 17:30	⚙️ 🗑️
LIVE	Belgium vs Iran	0 : 0	21 Jun 2026, 20:00	⚙️ 🗑️
SCHEDULED	New Zealand vs Egypt		in 2026, 02:00	⚙️ 🗑️
SCHEDULED	Argentina vs Austria		in 2026, 18:00	⚙️ 🗑️
SCHEDULED	Jordan vs Algeria		in 2026, 04:00	⚙️ 🗑️
SCHEDULED	France vs Iraq		in 2026, 22:00	⚙️ 🗑️
SCHEDULED	Norway vs Senegal		in 2026, 01:00	⚙️ 🗑️
SCHEDULED	Portugal vs Uzbekistan		Jun 2026, 18:00	⚙️ 🗑️
SCHEDULED	Colombia vs DR Congo	-	24 Jun 2026, 03:00	⚙️ 🗑️
SCHEDULED	England vs Ghana	-	23 Jun 2026, 21:00	⚙️ 🗑️
SCHEDULED	Panama vs Croatia	-	24 Jun 2026, 00:00	⚙️ 🗑️
SCHEDULED	Switzerland vs Canada	-	24 Jun 2026, 20:00	⚙️ 🗑️

New match

Home team
Select home team ▾

Away team
Select away team ▾

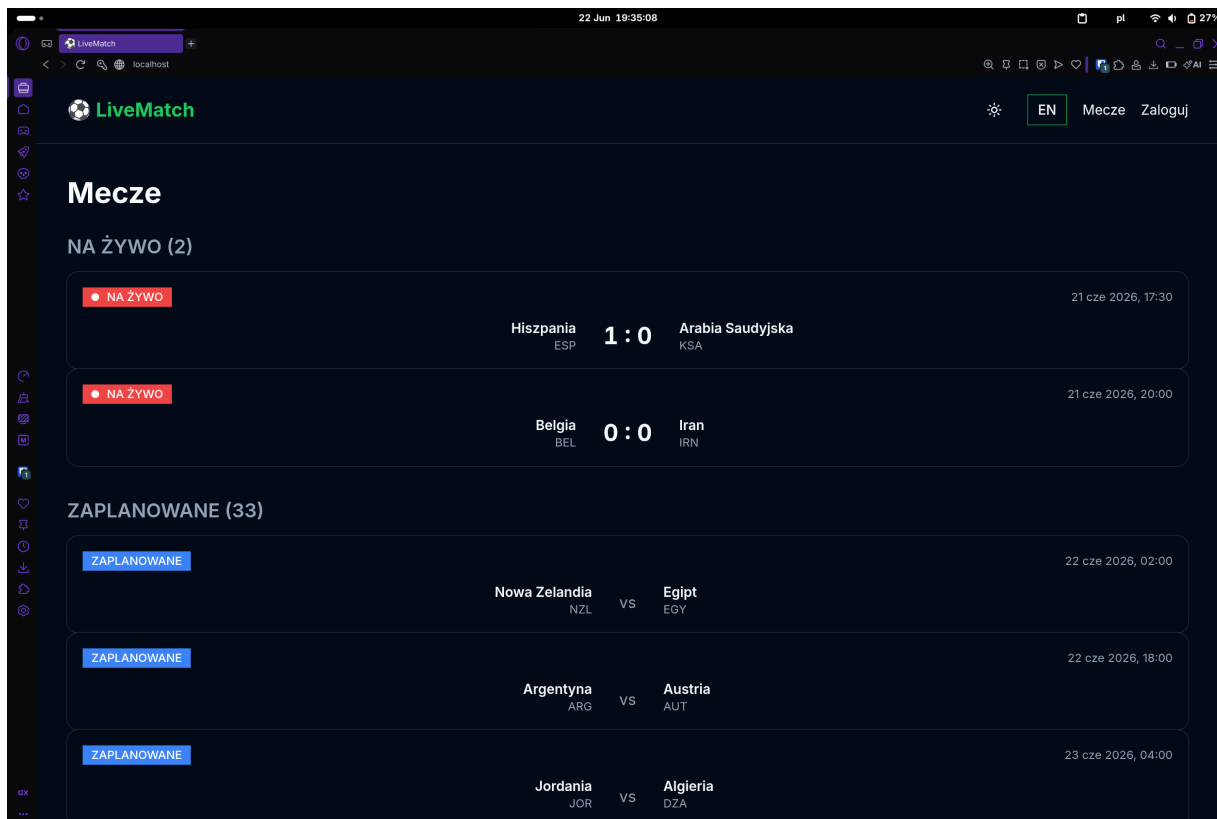
Start time
22/06/2026, 20:00 📅

Cancel Create match

Rysunek 13: Formularz rejestracji zdarzenia boiskowego

13.7 Zmiana języka i motywu

Przełącznik języka (PL/EN) oraz motywu (jasny/ciemny) dostępne są w prawym górnym rogu w pasku nawigacyjnym. Wybór jest zapamiętywany w `localStorage` przeglądarki (rys. 14).



Rysunek 14: Aplikacja w trybie ciemnym (dark) oraz w języku polskim

14 Informacja wdrożeniowa

14.1 Wymagania środowiska

Do uruchomienia aplikacji na maszynie docelowej konieczne jest:

- Docker w wersji ≥ 20.10 ,
- Docker Compose (v1 lub v2),
- Dostęp sieciowy do bazy danych AGH (`pascal.fis.agh.edu.pl:5432`) – w praktyce maszyna musi znajdować się w sieci wewnętrznej AGH,
- Wolny port **8090** dla frontendu.

14.2 Procedura wdrożenia

1. Połączenie z maszyną wirtualną AGH przez RDP (przez serwer dostępowy *Taurus*).
2. Klonowanie repozytorium:

```
1 git clone https://github.com/ur8an5ky/LiveMatch.git
2 cd LiveMatch
```

3. Uruchomienie kompozycji Docker:

```
1 docker-compose up -d --build
```

4. Po zakończeniu buildu (3-7 minut przy pierwszym uruchomieniu) aplikacja jest dostępna pod adresem `http://172.20.41.52:8090` (w sieci wewnętrznej AGH).

14.3 Zatrzymanie i sprzątnie

Standardowe polecenia zarządzające kontenerami:

```
1 docker-compose down          # zatrzymanie i usunięcie kontenerów
2 docker-compose logs -f       # podgląd logów na żywo
3 docker-compose ps            # status kontenerów
```

14.4 Domyślne dane logowania

Przy pierwszym uruchomieniu aplikacja tworzy automatycznie konto administratora:

- login: `admin`
- hasło: `admin123`

W środowisku produkcyjnym należy te dane natychmiast zmienić.

15 Wnioski

15.1 Realizacja wymagań projektowych

Projekt *LiveMatch* w pełni realizuje wymagania stawiane projektom zaliczeniowym:

- Aplikacja klient-serwer w architekturze REST – **zrealizowane** przez kontrolery Spring REST.
- Komunikacja WebSocket – **zrealizowane** (Spring WebSocket + STOMP, własne kanały tematyczne).
- Konteneryzacja serwera i bazy – **zrealizowane** (Docker Compose, multi-stage build).
- Spring Boot + Spring Data JPA – **zrealizowane**.
- Programowanie aspektowe – **zrealizowane** (Spring AOP, własna adnotacja `@Auditable`, aspekt audytujący).
- Klient SPA – **zrealizowane** (React + Vite).
- Diagramy UML – **zrealizowane** (przypadków użycia, klas, ERD, sekwencji).
- Testy jednostkowe i integracyjne – **zrealizowane** (20 testów backendowych).
- Dokumentacja – **zrealizowane** (niniejszy dokument).
- JavaDoc – **zrealizowane** (`mvn javadoc:javadoc`).
- Podręcznik użytkownika – **zrealizowane** (sekcja 12).
- Informacja wdrożeniowa – **zrealizowane** (sekcja 13 i `README.md`).

15.2 Wyzwania i decyzje projektowe

Podczas implementacji najczęściej uwagi wymagały następujące zagadnienia:

- **Konfiguracja CORS w środowisku produkcyjnym** – pierwotnie skonfigurowane dla portu deweloperskiego Vite (5173) ograniczenia musiały zostać uogólnione na dowolny port (`allowedOriginPatterns("*")`), aby działały zarówno z portu 80 (lokalnie), jak i 8090 (AGH).
- **Routing nginx dla WebSocket** – konieczna była precyzyjna konfiguracja `location /ws` (bez ukośnika końcowego) z nagłówkami `Upgrade` i `Connection`, aby umożliwić przejście protokołu HTTP na WebSocket przez reverse proxy.
- **Identyfikacja reprezentacji narodowych** – przyjęto konwencję, w której pole `name` encji `Team` pozostaje `NULL` dla reprezentacji (identyfikowanych kodem ISO), a wypełnione dla klubów. Rozwiązanie to upraszcza model bez dodawania pola dyskryminującego.

15.3 Kierunki rozwoju

W przypadku dalszego rozwoju projektu naturalne wydają się następujące rozszerzenia:

- Wprowadzenie rejestracji i logowania użytkowników, a co za tym idzie możliwości subskrypcji meczów i drużyn.
- Wprowadzenie wielowątkowości w aspekcie audytu (asynchroniczne zapisy w osobnym wątku, aby nie blokować logiki biznesowej).
- Obsługa większej liczby ról – np. `ROLE_REPORTER` z ograniczonymi uprawnieniami w stosunku do `ROLE_ADMIN`.
- Integracja z zewnętrznym API dostawcy wyników (np. `football-data.org`) jako alternatywne źródło danych.
- Migracja schematu bazy na narzędzie wersjonujące (Flyway lub Liquibase) zamiast `spring.jpa.hibernate.ddl-auto=update`.
- Wdrożenie progresywnej aplikacji webowej (PWA) z powiadomieniami push.